

Assignment 3: Bridge Pattern Report

Student Name: Sarsenbekov Sanzhar

Group: SE-2524

Topic: Drawing

1. System Overview

This system models a geometric shape rendering application designed around two independently varying dimensions:

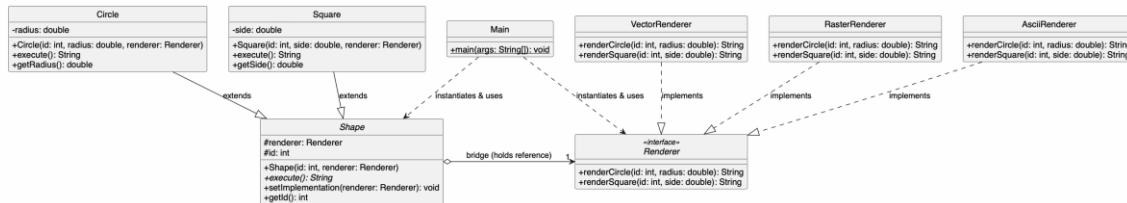
1. **Abstraction Dimension:** Geometric shapes (Shape, Circle, Square).
2. **Implementor Dimension:** Rendering media (Renderer, VectorRenderer, RasterRenderer, AsciiRenderer).

Why Bridge Fits

Without the Bridge pattern, combining N shapes with M renderers via standard class inheritance would cause a **Class explosion** ($N \times M$), requiring classes like VectorCircle, RasterCircle, AsciiCircle, VectorSquare, etc..

By applying composition over inheritance, the **Bridge Pattern** decouples the Shape abstraction hierarchy from the Rendererimplementor hierarchy. The Shape base class maintains a reference interface to Renderer. This allows shapes and renderers to vary independently, reducing total class complexity from $N \times M$ to $N + M$.

2. Structural UML Class Diagram



3. Clean Code Excerpts & Benefits

Excerpt 1: Separation of Responsibilities

Explanation & Benefit: Shape manages core domain attributes (id, renderer composition link) while leaving low-level graphics execution to Renderer implementations. High-level domain logic remains separate from rendering algorithms.

```
public abstract class Shape{ 10 usages 2 inheritors 2 PageHun
    private final int id; 2 usages
    protected Renderer renderer; 4 usages
    public Shape(int id, Renderer renderer){ 9 usages 2 PageHun
        this.id = id;
        this.renderer = renderer;
    }

    public int getId(){ 2 PageHun
        return id;
    }

    public void setImplementation(Renderer renderer) { this.renderer = renderer; }

    public abstract String execute(); 8 usages 2 implementations 2 PageHun
}
```

Excerpt 2: Meaningful Names

Explanation & Benefit: The method name `renderShape` and its parameter identifiers (`shapeType`, `id`, `dimension`) clearly express intent without requiring internal comments.

```
public interface Renderer { 13 usages  ⓘ PageHun
    public String renderCircle(int id, double radius); 1 usage 3 implementations  ⓘ PageHun
    public String renderSquare(int id, double side); 1 usage 3 implementations  ⓘ PageHun
}
```

Excerpt 3: Small Focused Methods

Explanation & Benefit: The `execute()` method does exactly one thing: delegates dimension-specific data to the implementation bridge. It contains no branching, rendering logic, or type casting.

```
@Override 8 usages  ⓘ PageHun
public String execute() { return renderer.renderCircle(getId(), radius); }
```

Excerpt 4: Avoidance of Duplication

Explanation & Benefit: Common fields (`id`, `renderer`), constructor initialization, getter logic, and the `setImplementation` method are defined once in the abstract base class `Shape`. Subclasses (`Circle`, `Square`) inherit this functionality directly, eliminating duplicated state management and setter/getter boilerplate across the abstraction hierarchy.

```
public abstract class Shape { 10 usages 2 inheritors  ⓘ PageHun
    private final int id; 2 usages
    protected Renderer renderer; 4 usages
    public Shape(int id, Renderer renderer) { 9 usages  ⓘ PageHun
        this.id = id;
        this.renderer = renderer;
    }

    public int getId() { ⓘ PageHun
        return id;
    }

    public void setImplementation(Renderer renderer) { this.renderer = renderer; }

    public abstract String execute(); 8 usages 2 implementations  ⓘ PageHun
}
```

Excerpt 5: Encapsulation of State

Explanation & Benefit: The field `radius` is declared private and final, preventing unintended mutation while safely exposing state via getter methods.

```
public abstract class Shape{ 10 usages 2 inheritors 2 PageHun
    private final int id; 2 usages
    protected Renderer renderer; 4 usages
    public Shape(int id, Renderer renderer){ 9 usages 2 PageHun
        this.id = id;
        this.renderer = renderer;
    }

    public int getId(){ 2 PageHun
        return id;
    }

    public void setImplementation(Renderer renderer) { this.renderer = renderer; }

    public abstract String execute(); 8 usages 2 implementations 2 PageHun
}
```

4. Extension, Trade-Off, and Pattern Comparison

Addition of I3 (AsciiRenderer)

To add the new implementation `AsciiRenderer` (I3), a single class implementing `Renderer` was created. The existing base class `Shape`, concrete refined abstractions (`Circle`, `Square`), and existing renderers (`VectorRenderer`, `RasterRenderer`) remained 100% unchanged. Only `Main.java` was updated to register tests T6 and T7. This demonstrates full compliance with the Open/Closed Principle.

Trade-Off of the Bridge Pattern

- **Increased Structural Complexity:** The Bridge pattern introduces extra abstraction layers and interfaces, which can slightly increase initial

architectural complexity for small projects that may not expand across multiple dimensions.

Bridge vs. Adapter Pattern

- **Bridge (Intent):** Designed **up-front** during architectural design to allow two independent dimensions to vary freely through composition without class explosion.
- **Adapter (Intent):** Applied **retroactively** to convert an existing, incompatible interface into another interface expected by a client, making unrelated classes work together without modifying their legacy source code.

References

- Ospanov A. (2026). Lecture 4: The Structural Design pattern – Bridge pattern Astana IT University.
- Ospanov A. (2026). Lecture 3: Structural Design Pattern – The Adapter Astana IT University.
- GeeksforGeeks. (2024, February 21). *Difference between Bridge Pattern and Adapter Pattern*. Retrieved October 2, 2026, from <https://www.geeksforgeeks.org/system-design/difference-between-bridge-pattern-and-adapter-pattern/>